

PHASE 6 – PROJECT TESTING PHASE

Project Title: Edugenie – Google Gemini Powered Learning Assistant

6.1 Introduction

The Project Testing Phase is an important stage in the development of Edugenie. The purpose of this phase is to verify that the learning assistant works correctly according to the requirements identified in the earlier project phases. Testing is performed to identify errors, incorrect outputs, integration problems, usability issues, and unexpected system behavior.

Edugenie is designed to support students by allowing them to enter learning-related questions and receive AI-generated explanations through Google Gemini. Since the application depends on user input, application logic, and an external AI service, each of these components must be tested carefully.

6.2 Purpose of Testing

- Verify that the major features of Edugenie operate correctly.
- Check whether user questions are accepted and processed properly.
- Verify the integration between the application and Google Gemini.
- Identify incorrect behavior and technical errors before demonstration.
- Check how the application handles invalid, empty, or unexpected inputs.
- Evaluate usability, response display, and basic performance.

6.3 Testing Scope

Testing covers the user interface, question submission, response generation, API integration, error handling, navigation, and basic performance. The testing process focuses on whether the system behaves according to its intended requirements rather than judging the factual quality of every possible AI response.

6.4 Testing Objectives

The testing objectives define what the project team expects to verify before the application is considered ready for demonstration.

- To confirm that the application starts and loads without errors.
- To confirm that users can enter learning questions successfully.
- To verify that submitted questions reach the intended processing component.
- To verify that a response is displayed after successful processing.
- To check successful communication with the Google Gemini API.
- To verify suitable handling of network or API failures.
- To test empty and invalid input conditions.
- To check that repeated questions can be submitted without application failure.
- To verify that the interface remains understandable and easy to use.
- To identify defects and record the actions taken to correct them.

6.5 Testing Environment

Component	Testing Environment
Application	Edugenie – Google Gemini Powered Learning Assistant
Client	Web/application interface used for the project
AI Service	Google Gemini API
Input	Student learning questions and test inputs
Network	Internet connection required for API-based responses
Testing Type	Functional, integration, input validation, usability and basic performance testing

6.6 Test Data

Test data consists of normal educational questions, short questions, longer questions, empty input, repeated questions, unrelated text, and inputs that may cause an API or network error. Using different types of input helps determine whether the application behaves consistently under common usage conditions.

Examples include questions about programming concepts, definitions, explanations, examples, and general academic topics. The same application flow is tested with different input lengths to observe the response behavior.

6.7 Testing Methodologies

6.7.1 Functional Testing

Functional testing checks whether each required feature produces the expected behavior. For Edugenie, this includes opening the application, entering a question, submitting the question, receiving a response, and displaying the response to the user.

6.7.2 Integration Testing

Integration testing verifies communication between the user interface, application logic, and Google Gemini API. A successful integration should allow a submitted question to be processed and the returned response to be displayed correctly.

6.7.3 Input Validation Testing

Input validation testing checks how the system behaves when users provide empty, incomplete, unusually long, or unexpected input. The application should not crash because of an invalid input.

6.7.4 Usability Testing

Usability testing checks whether the application is simple enough for students to understand. Labels, input areas, buttons, response areas, and navigation should be clear and usable.

6.7.5 Performance Testing

Basic performance testing observes whether the application responds within a reasonable time under normal conditions. Since AI response time can depend on network and service conditions, the test focuses on whether the application remains responsive and communicates errors appropriately.

6.7.6 Error Handling Testing

Error handling testing verifies that API failures, network interruptions, invalid responses, and unexpected conditions are handled without causing the application to terminate unexpectedly.

6.8 Testing Process

- Prepare the test environment.
- Start the Edugenie application.
- Enter a defined test input.
- Submit the input and observe system behavior.
- Compare the observed result with the expected result.
- Record Pass or Fail status.

- Identify the cause of failed cases.
- Apply corrections and repeat the relevant test.

6.9 Functional Test Cases – Part I

ID	Test Scenario	Test Input/Action	Expected Result	Status
TC01	Application launch	Open Edugenie	Application opens successfully	Pass
TC02	Input field	Enter a valid question	Question is accepted	Pass
TC03	Submit question	Click submit/send	Question is processed	Pass
TC04	Response display	Submit valid question	AI response is displayed	Pass
TC05	Second query	Submit another question	New response is generated	Pass
TC06	Short input	Enter a short academic question	System processes input	Pass
TC07	Long input	Enter a detailed question	System accepts and processes input	Pass
TC08	Repeated query	Submit the same type of question twice	Application continues working	Pass

6.10 Functional Test Cases – Part II

ID	Test Scenario	Test Input/Action	Expected Result	Status
TC09	Empty input	Submit blank input	Input is handled without crash	Pass
TC10	Unexpected text	Enter unrelated text	Application remains stable	Pass
TC11	Response area	Generate a response	Response appears in the correct area	Pass
TC12	Navigation	Move between available interface sections	Navigation works correctly	Pass
TC13	API request	Send valid query with network	Request reaches AI service	Pass
TC14	API response	Receive valid Gemini response	Response is shown to user	Pass

TC15	Multiple submissions	Send multiple queries sequentially	Application remains stable	Pass
------	----------------------	------------------------------------	----------------------------	------

The functional test cases are designed around the normal user journey. A test is marked Pass when the observed behavior matches the expected behavior. If a test fails, the issue is recorded, corrected, and retested.

6.11 Integration and Error Handling Test Cases

ID	Scenario	Condition	Expected Result	Status
IT01	Gemini API connectivity	Valid question + active network	API request is processed	Pass
IT02	API response handling	Valid API response	Response is displayed correctly	Pass
IT03	Network interruption	Disconnect/interrupt network	Application handles failure safely	Pass
IT04	API failure	Simulated service/API error	Suitable error handling is shown	Pass
IT05	Invalid/empty response	Unexpected service response	Application does not crash	Pass
IT06	Repeated API requests	Submit several questions	Requests are handled without interface failure	Pass

6.12 Input Validation Test Cases

ID	Scenario	Input Type	Expected Result	Status
IV01	Blank input	No text entered	System prevents or safely handles submission	Pass
IV02	Single-word input	One academic keyword	System processes input	Pass
IV03	Normal question	Complete academic question	Relevant response is returned	Pass
IV04	Long question	Detailed question	Input is handled without UI failure	Pass
IV05	Special characters	Symbols included in input	Application remains stable	Pass
IV06	Repeated input	Same question submitted again	Application continues functioning	Pass

6.13 Usability Checks

- Question input area is clearly identifiable.
- Submit/send action is easy to understand.
- Generated responses are visible to the user.
- Interface does not require unnecessary steps for a basic query.
- Error messages, when required, should be understandable.
- Users can continue asking questions after receiving a response.

6.14 Defect Identification and Bug Tracking

During testing, defects are recorded so that the development team can identify the cause and apply corrections. A defect may relate to the interface, input processing, API communication, response display, or error handling.

Bug ID	Observed Issue	Possible Cause	Corrective Action	Status
BUG01	Response not displayed	Response handling/display issue	Check response processing and UI binding	Resolved
BUG02	Empty input causes unexpected behavior	Missing input validation	Add validation before submission	Resolved
BUG03	API request fails during network issue	Network/API dependency	Add error handling and user message	Resolved
BUG04	Unexpected input affects normal flow	Insufficient input handling	Improve validation and exception handling	Resolved
BUG05	Repeated query does not refresh response	State/update issue	Correct response update logic	Resolved

6.15 Bug Fixing Process

- Identify the problem from a failed test case.
- Record the exact observed behavior.
- Determine the likely source of the defect.
- Modify the relevant application component.
- Run the same test case again.
- Check whether the correction affects other features.
- Mark the defect as resolved only after successful retesting.

6.16 Retesting

Retesting is performed after a defect is corrected. The failed test case is executed again to verify that the original problem has been removed. Related test cases are also checked to ensure that the correction has not introduced a new problem.

For Edugenie, retesting is particularly important for API communication and response handling because changes in request or response processing can affect the main learning-assistant workflow.

6.17 Test Summary

Testing Area	Cases/Checks	Result
Application launch	Basic startup	Passed
User input	Valid, empty and unexpected input	Passed
Question submission	Normal and repeated queries	Passed
Gemini integration	Request and response flow	Passed
Error handling	API/network failure conditions	Passed
Response display	Generated response visibility	Passed
Usability	Input, button and response flow	Passed
Basic performance	Normal response behavior	Passed

6.18 Overall Test Result

The testing activities covered the major functional and supporting components of Edugenie. The application workflow was checked from user input through response generation and display. Input validation and error handling were also considered to reduce the possibility of application failure during normal use.

The recorded test cases are marked as Pass after the expected behavior was verified. Any identified defects were considered for correction and retesting. The test results provide evidence that the project is ready to proceed to the documentation and demonstration stages, subject to the final project environment and API availability.

6.19 Testing Limitations

- AI-generated responses can vary because they are produced dynamically.
- Response time may depend on internet connectivity and external API availability.
- Testing cannot cover every possible user question or input combination.
- External service changes may affect API behavior after project testing.
- Factual correctness of every generated AI response requires separate content evaluation and cannot be guaranteed by functional testing alone.

6.20 Acceptance Criteria

- The application should launch without critical errors.
- Users should be able to submit learning questions.

- The application should process valid questions through the intended AI service.
- Responses should be displayed to the user.
- Invalid or empty inputs should be handled safely.
- Major API and network failures should not crash the application.
- Critical defects should be corrected before demonstration.

6.21 Final Testing Checklist

Checklist Item	Verification	Status
Application starts correctly	Verified	Completed
User can enter a question	Verified	Completed
Question submission works	Verified	Completed
Gemini API integration works	Verified	Completed
Response is displayed	Verified	Completed
Empty input is handled	Verified	Completed
API/network error handling is considered	Verified	Completed
Repeated queries are supported	Verified	Completed
Basic usability is checked	Verified	Completed
Defects are recorded and retested	Verified	Completed

6.22 Final Test Report

The Project Testing Phase of Edugenie was carried out to verify the functionality, integration, usability, input handling, and basic performance of the learning assistant. The test cases were designed to represent common student interactions as well as error conditions.

Testing confirmed the intended application flow: the user enters a learning question, the system processes the request through the configured AI service, and the generated response is presented to the user. Input validation and error-handling scenarios were also included to improve application stability.

6.23 Conclusion

Testing is an essential step in ensuring that Edugenie is suitable for its intended purpose as a Google Gemini powered learning assistant. The testing process helps identify defects before final submission and demonstration. Based on the documented test cases and retesting activities, the major project functions have been verified.

The completed testing results support moving the project to the next phase, Project Documentation, where the development process, design, implementation, testing results, and project outcomes are formally recorded.

6.24 Phase 6 Completion

PROJECT TESTING PHASE – COMPLETED